

Operaciones de Aprendizaje Automático III

Alineación de LLM con Reinforcement Learning

MSc. Fernando Silva



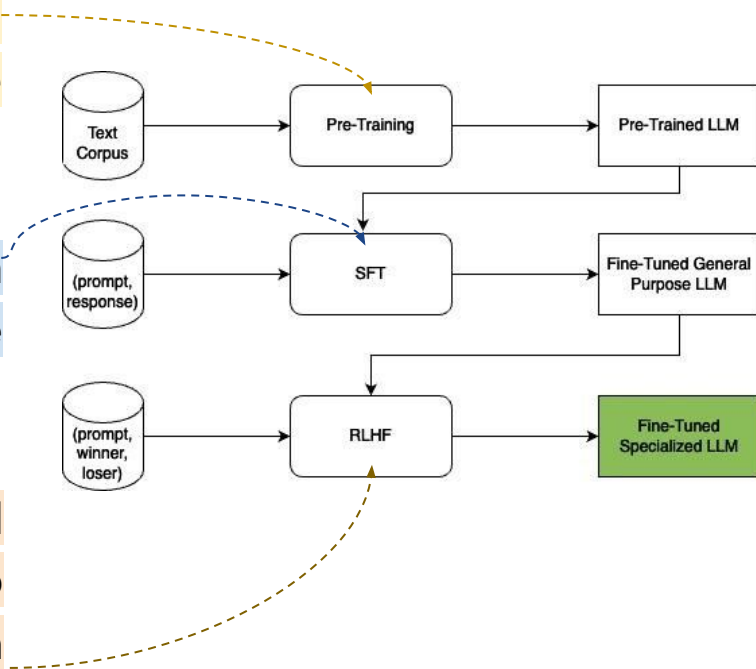
Agenda

- Group Relative Policy Optimization (GRPO)
- vLLM
- El juez como modelo servido
- Alineación de LLM con Reinforcement Learning

Group Relative Policy Optimization (GRPO)

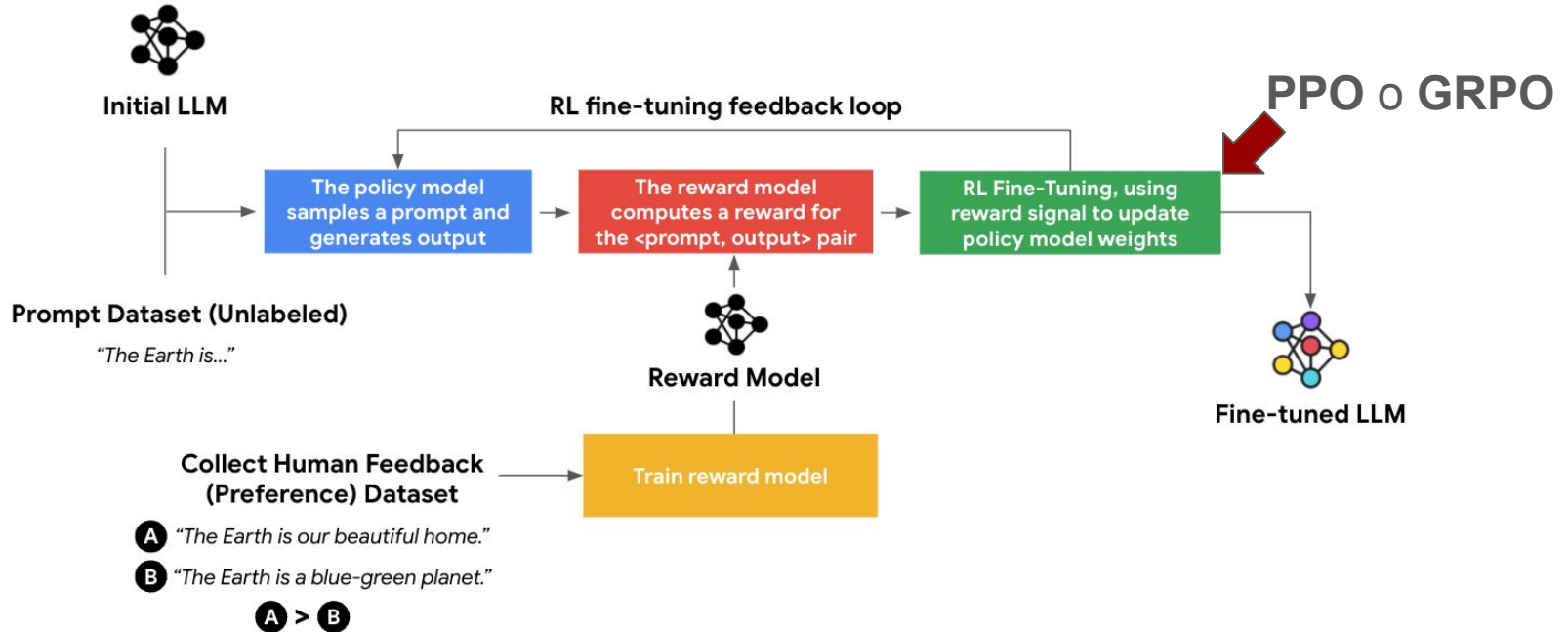
Flujo LLM

- Un LLM moderno pasa por varias etapas:
 - **Preentrenamiento:** lee enormes cantidades de texto y aprende a predecir la siguiente palabra. Así aprende lenguaje y conocimiento general.
 - **Ajuste supervisado (SFT):** se le enseñan ejemplos de buenas respuestas para que aprenda a seguir instrucciones.
 - **Aprendizaje por refuerzo (RL):** el modelo genera respuestas, se le premia o castiga según su calidad, y así mejora más allá de solo imitar ejemplos.



RLHF

RLHF (Reinforcement Learning from Human Feedback) es un método para alinear un modelo con las preferencias humanas. Se usaba PPO porque permite ajustar el modelo mediante recompensas manteniendo controladas las actualizaciones para evitar que se aleje demasiado del modelo original.

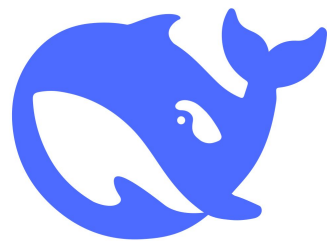


RLHF

- ChatGPT popularizó el RLHF (aprendizaje por refuerzo con retroalimentación humana):
- Personas comparaban respuestas y se entrenaba un "modelo de recompensa" que imitaba sus preferencias.
- Luego se usaba PPO para optimizar el LLM según esa recompensa.
- El problema es que PPO es pesado: requiere mantener varios modelos grandes en memoria (el LLM, el crítico, el modelo de recompensa y el modelo de referencia). Es caro e inestable. Por eso surgieron alternativas más simples, como DPO, que aprende directamente de pares de respuestas "mejor/peor" sin hacer RL propiamente dicho.

GRPO

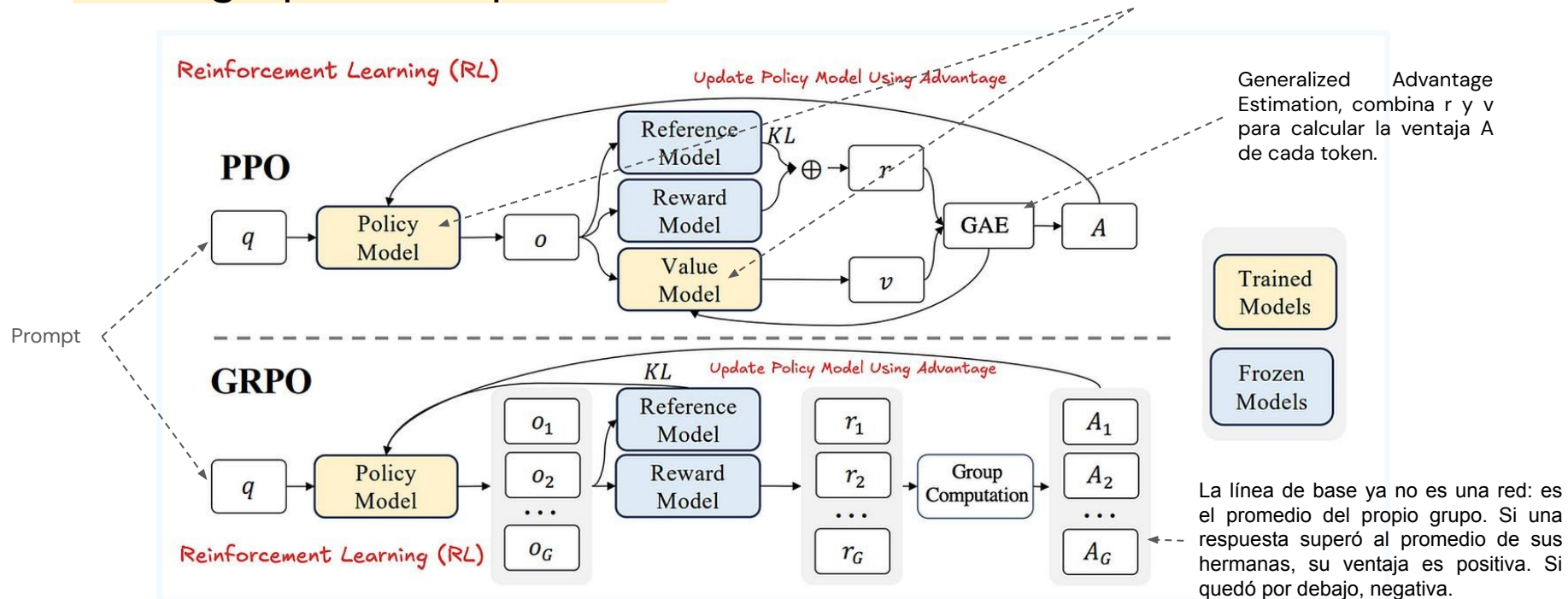
- DeepSeek propuso GRPO en 2024 (paper DeepSeekMath). Quitó el crítico de PPO y lo reemplazó por la comparación dentro de un grupo de respuestas. El resultado es RL real, pero mucho más barato que PPO.
- Sirve para mejorar un modelo de lenguaje premiando sus buenas respuestas, sobre todo en tareas donde se puede verificar si la respuesta es correcta, como matemáticas o código.



GRPO vs PPO

- Quitó el crítico de PPO y lo reemplazó por la comparación dentro de un grupo de respuestas.

Dos modelos entrenándose al mismo tiempo



GRPO

- Queremos que un loro aprenda a decir “hola”, pero no le podemos decir que palabra usar.
- Solo podemos darle una semilla cuando hace algo bien

Ronda 1:

intento	dice	semilla
1	"chau"	0
2	"hola"	1
3	"pío"	0
4	"qué tal"	0

El promedio del grupo es 1 semilla de 4, o sea 0.25. Entonces:

- "hola" sacó más que el promedio, así que la próxima vez lo dice más seguido
- "chau", "pío" y "qué tal" sacaron menos que el promedio, así que los dice un poco menos

Ronda 2:

intento	dice	semilla
1	"hola"	1
2	"hola"	1
3	"pío"	0
4	"hola"	1

El promedio sube a 0.75. "hola" sigue por encima y se refuerza, "pío" queda por debajo y se debilita.

beta es una multa por cambiar demasiado su forma de hablar

Ronda 10:

Dice "hola" las 4 veces y saca 4 semillas, el promedio es 1 y todos están igual que el promedio, así que no hay nada que corregir, ya aprendió.

A green parrot with a black beak and a white ring around its eye, standing on a white background. The parrot is facing left and has its wings slightly tucked.

Ejercicio GRPO

[LINK](#)

Por qué GRPO

Método	Qué exige	Por qué no para el proyecto
PPO	Un modelo de valor entrenado en paralelo	Duplica la memoria; en una T4 no entra
DPO	Pares etiquetados (elegido, descartado)	No hay preferencias, hay respuesta verificable
GRPO	Una función que devuelva un número	Es lo que ya tenemos escrito en Python

Cómo configuramos GRPO

```
GRPOConfig(  
    beta = 0.1 | 0.001,      # la única variable del experimento  
    num_generations = 8,     # el tamaño del grupo  
    generation_batch_size = lote_prompts * tamano_grupo,  
    max_steps = 200,  temperature = 1.0,  
    use_vllm = True,  vllm_mode = "colocate",  
    save_strategy = "no",    # los checkpoints los guarda un callback  
)
```

beta es el experimento

Pesa la divergencia KL contra la referencia. Dos brazos: 0.1 contiene, 0.001 casi no.

vLLM

vLLM

vLLM es una librería de código abierto para ejecutar LLMs de forma rápida y eficiente para generar respuestas (inferencia). Nació en la Universidad de California, Berkeley, en 2023, y es una de las herramientas más usadas para servir modelos.



vLLM

Entrenar un LLM es caro, pero usarlo a gran escala también lo es. Cuando miles de usuarios piden respuestas al mismo tiempo:

- La GPU se queda sin memoria rápidamente.
- Muchas peticiones esperan en cola.
- Se desperdicia mucha capacidad del hardware.

Las herramientas básicas, como generar con Hugging Face Transformers, funcionan bien para pruebas, pero son lentas para producción.

Al generar texto, el modelo guarda en memoria información de las palabras anteriores. Esa memoria se llama KV cache y crece con cada palabra generada.

Antes se reservaba un bloque grande y fijo de memoria por cada petición "por si acaso". Así se desperdiciaba mucha memoria que nunca se usaba.

vLLM

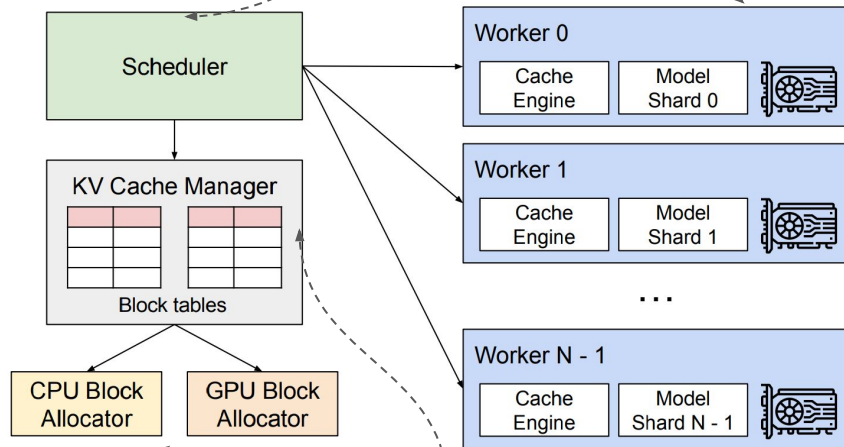
Paged Attention: vLLM tomó una idea de los sistemas operativos (la memoria virtual paginada):

- Divide la KV cache en bloques pequeños.
- Los asigna solo cuando se necesitan.
- Peticiones que comparten el mismo inicio, como un mismo prompt de sistema, pueden compartir bloques.

Así casi no se desperdicia memoria y caben muchas más peticiones a la vez.

Este es el orquestador, en cada paso de generación decide qué secuencias entran. No ejecuta nada, decide.

es para cuando el modelo no entra en una sola GPU, lo partimos en N pedazos y cada GPU se encarga de uno



cada secuencia tiene una tabla de bloques: una lista que dice "mis tokens 0-15 están en el bloque 7 de la GPU, los 16-31 en el bloque 23..."

este es el pozo de bloques libres, hay uno en GPU y otro en CPU, si la GPU se llena, los bloques de una secuencia pueden mandarse temporalmente a RAM y traerse de vuelta después

vLLM vs transformers

- transformers es la librería con la que se entrena y se experimenta
- vLLM es el motor con el que se sirve

Las dos generan texto idéntico, lo que cambia es cuántas secuencias caben a la vez en la GPU.

	transformers	vLLM
Qué es	Librería de propósito general	Motor de inferencia
Entrena	Sí	No
Genera texto	Sí	Sí (el mismo texto)
Cómo arma el lote	Tamaño fijo: todas arrancan y terminan juntas	Continuo: entran y salen en cualquier paso
Memoria de la caché	Reserva el peor caso y rellena hasta el más largo	Bloques de 16 tokens, se piden a medida que se usan
Si una secuencia termina antes	Sigue ocupando lugar hasta que termine el lote	Libera sus bloques al instante
Pedidos concurrentes	El segundo espera a que termine el primero	Conviven en el mismo paso
Varios LoRA sobre una base	No: hay que fusionar y cargar un modelo por variante	Sí: una copia de la base, se elige por nombre en el pedido
Trae servidor HTTP	No, hay que escribirlo	Sí, compatible con la API de OpenAI
Para qué usarla	Entrenar, experimentar, debuggear, mirar adentro	Servir en producción y generar masivamente (RL)

vLLM

En GRPO, el modelo tiene que generar muchas respuestas por pregunta (el grupo), y esa generación es la parte más lenta del entrenamiento.

Por eso librerías como TRL, verl y OpenRLHF usan vLLM para generar las respuestas y otro motor para actualizar los pesos. Sin un motor de inferencia rápido como vLLM, el RL en LLMs sería mucho más lento y caro.

1

Genera los rollouts durante el entrenamiento

Modo colocate: comparte la GPU con el entrenamiento, sin proceso aparte.

2

Sirve el artefacto para evaluarlo

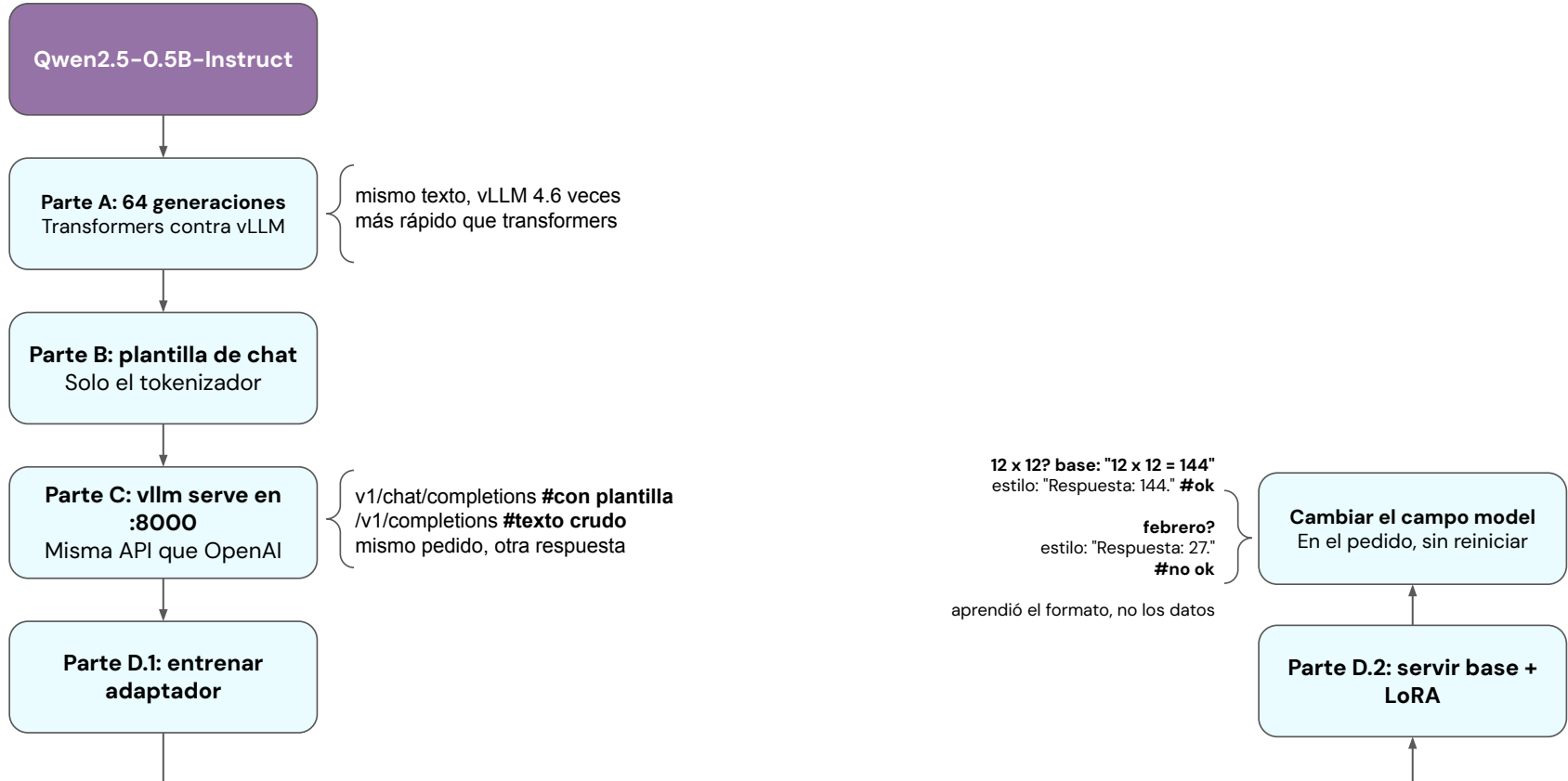
Puerto 8000. La base cargada una vez y los tres adaptadores registrados juntos.

3

Sirve el juez

Puerto 8001, otro modelo, proceso separado, deliberadamente.

Flujo vLLM



A green parrot with a black beak and a white ring around its eye is standing on a white background. The parrot is facing left and has its wings slightly tucked.

Ejercicio vLLM en tres papeles

[LINK](#)

El juez como modelo servido

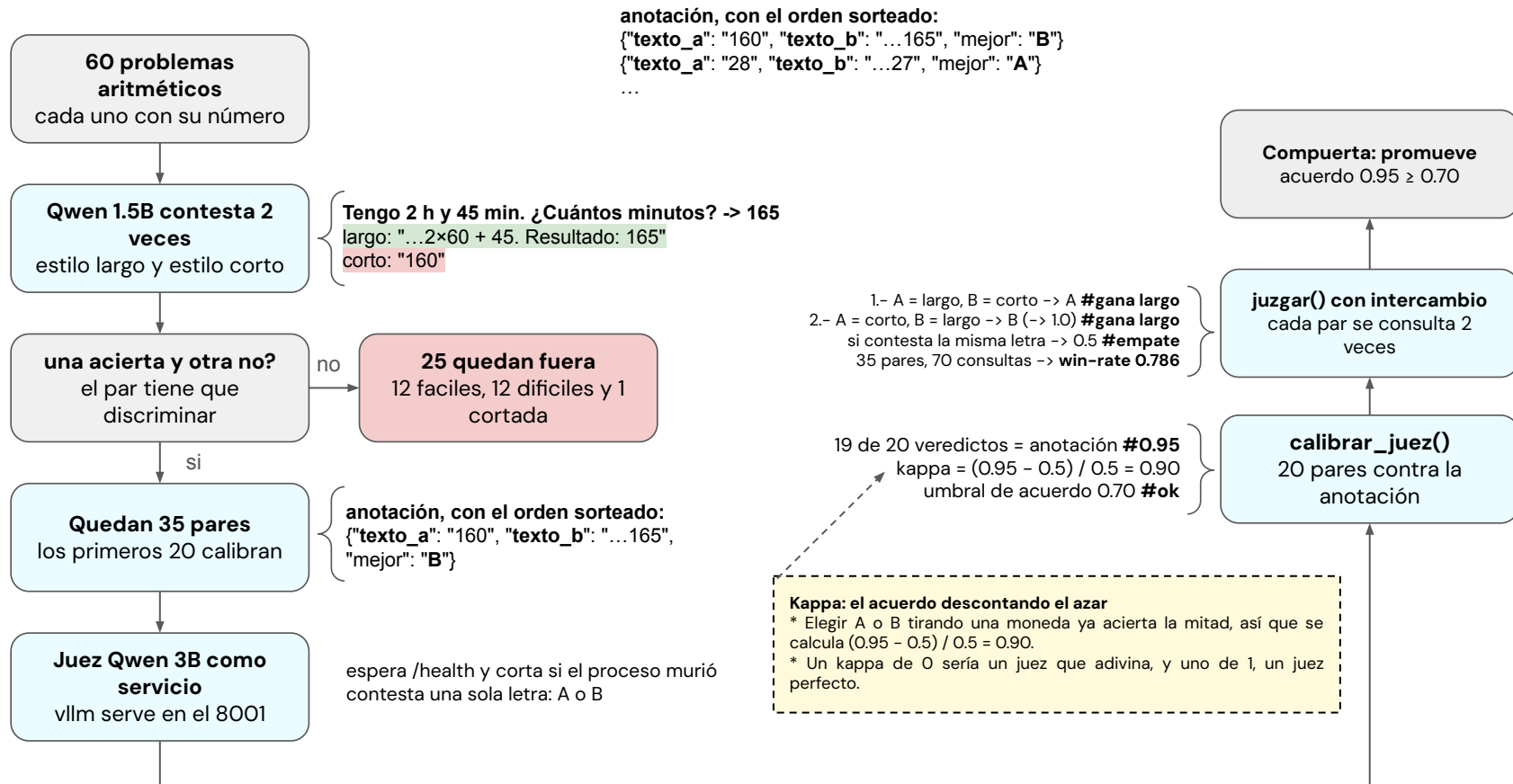
Por qué hace falta un juez

La recompensa mide lo que le pedimos. Usarla también para evaluar sería circular: el modelo que mejor la explota sería, por definición, el mejor modelo.

- Comparación pareada. No se pide una nota del 1 al 10, que ningún modelo da de forma estable: se pide elegir entre dos respuestas al mismo problema, la alineada y la de la base.
- Criterios en orden. El resultado correcto primero; después que el razonamiento se pueda seguir; al final, que no haya relleno.
- Se le da la respuesta correcta. El juez no resuelve el problema, solo compara: una tarea mucho más fácil y más confiable.

La métrica que sale de acá es el win-rate: en qué fracción de los 120 problemas prefirió la alineada.

Flujo del juez



Un juez sin calibrar no es una medición

Antes de usarlo se mide contra anotaciones hechas a mano.

Si no coincide con nosotros lo suficiente, no sirve para decidir nada.

$$\text{acuerdo} = \text{aciertos} / n$$
$$\text{kappa} = (\text{acuerdo} - 0.5) / 0.5$$

- Con dos categorías equiprobables el azar acierta la mitad. Kappa descuenta eso: mide cuánto mejor que una moneda es el juez.
- `umbral_acuerdo` = 0.70, fijado en el perfil antes de medir. No se ajusta después de ver el resultado.
- Es la primera regla de la compuerta: si el acuerdo no llega, nada se promueve, por bueno que se vea el win-rate.

El sesgo de posición

Los modelos tienden a preferir lo que leyeron primero. Si la respuesta alineada fuera siempre la A, parte del win-rate sería eso y nada más.

Orden aleatorio

Para cada problema se sortea cuál va primero, con semilla fija por corrida.

Se pregunta dos veces

La misma comparación con A y B intercambiadas. Dos consultas por problema.

Si se contradice

Coinciden las dos: vale 1 o 0.
Se contradice al invertir: 0.5, empate.

El empate no es un parche: un juez que se contradice al invertir el orden no tiene opinión sobre ese par, y registrarlo como medio punto es más honesto que forzar una preferencia.

Cuesta el doble de consultas. Es el gasto más grande del pipeline después del entrenamiento.

Alineación de LLM con RL

Contexto

¿Qué se alinea?

El proyecto alinea el modelo a una consigna escrita, que está en datos.py:

Resolvé el problema paso a paso, en no más de tres oraciones, y cerrá con el resultado numérico entre las etiquetas <respuesta> y </respuesta>.

Eso es todo lo que significa "alineado"

No se puede entrenar contra una consigna en castellano

Hay que traducirla a un número, y eso es recompensas.py:

la consigna pide	la recompensa mide	peso
cerrar con las etiquetas	¿está <respuesta>N</respuesta>?	0.3
resolver el problema	¿el número es el correcto?	0.5
en no más de tres oraciones	¿es corta?	0.9

Flujo

GRPO

LLM
OPs

1.- Contrato de datos
la recompensa es código
datos.py - recompensas.py

LLM
OPs

Reentrenamiento con lo
observado

LLM
OPs

Compuerta
Cuatro reglas, decide sola
compuerta.py

LLM

LLM
OPs

Evaluación
Juez calibrado, IC del 95%
evaluar.py - analizar.py

LLM

LLM
OPs

Serving
vLLM compone al desplegar
evaluar.py

LLM
GRPO

Entrenamiento
GRPO + LoRA, vLLM genera
alinear.py

LLM
OPs

Trazabilidad
Miflow por corrida y semilla
registro.py

LLM
OPs

Artefacto
Base + adaptador + entorno
entorno.py

Qué se registra en cada corrida

Qué	Contenido
Etiquetas	etapa, brazo, semilla, perfil. Son las que permiten filtrar las seis corridas
Parámetros	beta, el bloque de modelo, el de GRPO, el de recompensa y la huella del entorno
Métricas	Recompensa, largo, KL, clipped_ratio, frac_reward_zero_std y entropía, paso a paso
Artefactos	Los adaptadores de cada checkpoint y la figura del análisis

Los parámetros se guardan aplanados: el diccionario anidado se vuelve claves con punto.

El perfil entero queda recuperable desde el registro, sin confiar en que el YAML del repositorio no cambió desde entonces.

La base está en un SQLite dentro de la carpeta de artefactos. No hace falta servidor: mlflow ui lo abre local.